

DomaWearOS

Documentação do Trabalho Prático — DGT2816
Interação com sensores de smartphones e wearables

DISCIPLINA

DGT2816

PLATAFORMA

Wear OS

LINGUAGEM

Kotlin

INSTITUIÇÃO

Estácio

1. Contexto e Objetivo

A empresa fictícia **Doma** identificou a necessidade de fornecer assistência sonora a funcionários com deficiência auditiva ou visual no ambiente corporativo. O objetivo é desenvolver um aplicativo **Wear OS** (smartwatch) capaz de usar o áudio como canal principal de comunicação, entregando informações, alertas e feedbacks via fala sintetizada.

Funcionalidades entregues pelo app:

- 🔊 Leitura de mensagens de texto em voz alta (TTS)
- 🔔 Reprodução de notificações em áudio
- ⌚ Lembretes por voz em Português Brasileiro
- 📶 Detecção automática de saídas de áudio disponíveis
- 🎧 Suporte a fones Bluetooth A2DP e com fio
- 🚨 Alertas críticos de segurança (evacuação, incêndio, emergência médica)

2. Microatividade 1 — Configuração do Ambiente

2.1 Ferramentas Instaladas

Ferramenta	Versão	Finalidade
Android Studio	Hedgehog+	IDE principal de desenvolvimento
Kotlin	1.9.22	Linguagem de programação
Android SDK	API 34	Compilação e target
Wear OS SDK	API 30 (minSdk)	Plataforma wearable
Gradle	8.2	Build system

2.2 Emulador Wear OS

Configuração do emulador no Android Studio:

Tools → Device Manager → Create Device → Wear OS → Wear OS Small Round → API 30

2.3 Permissões — AndroidManifest.xml

Permissões declaradas no manifesto do aplicativo:

BODY_SENSORS WAKE_LOCK BLUETOOTH BLUETOOTH_CONNECT VIBRATE

```
<uses-permission android:name="android.permission.BODY_SENSORS" />
<uses-permission android:name="android.permission.WAKE_LOCK" />
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.VIBRATE" />
<uses-feature android:name="android.hardware.type.watch" />
```

3. Microatividade 2 — Implementação

3.1 AudioHelper.kt — Detecção de Saídas de Áudio

Classe utilitária que encapsula toda a lógica de detecção de dispositivos de saída de áudio. O método principal `audioOutputAvailable(type)` implementa o padrão exigido pela especificação:

```
fun audioOutputAvailable(type: Int): Boolean {
    // 1. Verifica se o dispositivo possui saída de áudio
    if (!context.packageManager
        .hasSystemFeature(PackageManager.FEATURE_AUDIO_OUTPUT)) {
        return false
    }
    // 2. Enumera saídas e verifica se o tipo solicitado existe
    return audioManager
        .getDevices(AudioManager.GET_DEVICES_OUTPUTS)
        .any { it.type == type }
}
```

Método	Tipo AudioDeviceInfo	Descrição
<code>isSpeakerAvailable()</code>	TYPE_BUILTIN_SPEAKER	Alto-falante integrado do relógio
<code>isBluetoothHeadsetConnected()</code>	TYPE_BLUETOOTH_A2DP	Fone de ouvido Bluetooth
<code>isWiredHeadsetConnected()</code>	TYPE_WIRED_HEADPHONES	Fone com fio

3.2 MainActivity.kt — Tela Principal

Implementa a `ListView` com as 9 funcionalidades assistivas e os 3 botões de acesso rápido:

```
val funcionalidades = listOf(
    "🔊 Leitura de mensagens de texto",
    "🔔 Notificações em áudio",
    "🗣️ Lembretes por voz",
    // ...
)
lvFuncionalidades.adapter = ArrayAdapter(
    this, android.R.layout.simple_list_item_1, funcionalidades
)
```

3.3 AudioDeviceCallback — Detecção Dinâmica em Tempo Real

Detecta conexão/desconexão de fones Bluetooth **sem precisar reiniciar o app**, atualizando o status na tela automaticamente:

```
private val audioDeviceCallback = object : AudioDeviceCallback() {
    override fun onAudioDevicesAdded(
        addedDevices: Array<out AudioDeviceInfo>?
    ) {
        if (audioHelper.isBluetoothHeadsetConnected())
            updateAudioStatus("🎧 Fone Bluetooth conectado!")
    }
    override fun onAudioDevicesRemoved(
        removedDevices: Array<out AudioDeviceInfo>?
    ) {
        if (!audioHelper.isBluetoothHeadsetConnected())
            updateAudioStatus("🔴 Bluetooth desconectado.")
    }
}
// Registra no onCreate()
audioManager.registerAudioDeviceCallback(audioDeviceCallback, null)
// Remove no onDestroy() — evita memory leak
audioManager.unregisterAudioDeviceCallback(audioDeviceCallback)
```

3.4 Intent para Configurações Bluetooth

Redireciona o usuário para parear fones de ouvido assistivos:

```
val intent = with(Intent(Settings.ACTION_BLUETOOTH_SETTINGS)) {
    addFlags(Intent.FLAG_ACTIVITY_NEW_TASK
        or Intent.FLAG_ACTIVITY_CLEAR_TASK)
    putExtra("EXTRA_CONNECTION_ONLY", true)
    putExtra("EXTRA_CLOSE_ON_CONNECT", true)
    putExtra("android.bluetooth.devicepicker.extra.FILTER_TYPE", 1)
    this
}
startActivity(intent)
```

3.5 AudioAssistActivity.kt — Text-to-Speech PT-BR

Converte texto em fala em Português Brasileiro, verificando a saída disponível antes de reproduzir:

```
// Inicializa TTS com PT-BR
tts = TextToSpeech(this, this)

override fun onInit(status: Int) {
    if (status == TextToSpeech.SUCCESS)
        tts.setLanguage(Locale("pt", "BR"))
}

private fun reproduzirAudio(texto: String) {
    when {
        audioHelper.isBluetoothHeadsetConnected() →
            tvStatus.text = "🎧 Reproduzindo via Bluetooth ..."
        audioHelper.isSpeakerAvailable() →
            tvStatus.text = "🔊 Reproduzindo via alto-falante ..."
        else → { /* sem saída — exibe Toast */ return }
    }
    tts.speak(texto, TextToSpeech.QUEUE_FLUSH, null, "doma_tts")
}
```

3.6 AlertActivity.kt — Alertas com Vibração

Emite 4 tipos de alerta crítico combinando TTS + padrão de vibração tátil:

Alerta	Mensagem TTS	Padrão Vibração
🚪 Evacuação	Dirija-se à saída de emergência!	3x longo (500ms)
🔥 Incêndio	Saia do prédio imediatamente!	3x médio (300ms)
🏥 Médico	Atendimento médico urgente!	2x curto (200ms)
🛡️ Segurança	Aviso geral da Doma	1x (100ms)

4. Estrutura de Arquivos do Projeto

```
DomaWearOS/ ├── .gitignore ├── README.md ├── build.gradle # config raiz do projeto
├── settings.gradle # inclui módulo :app ├── gradle.properties # flags do Gradle
├── gradle/wrapper/ | └── gradle-wrapper.properties # versão do Gradle (8.2) ├──
docs/ | └── documentacao_dgt2816.html # ← este documento └── app/ ├── build.gradle
# deps Wear OS, Kotlin, TTS └── src/main/ ├── AndroidManifest.xml # permissões,
activities, serviço ├── java/com/doma/wearos/presentation/ | ├── AudioHelper.kt #
audioOutputAvailable() | ├── MainActivity.kt # ListView, Buttons, Callback | ├──
AudioAssistActivity.kt# TTS PT-BR | ├── AlertActivity.kt # alertas + vibração | └──
AudioAssistService.kt# serviço background └── res/ ├── layout/ | ├──
activity_main.xml | ├── activity_audio_assist.xml | └── activity_alert.xml ├──
values/ | ├── strings.xml | └── colors.xml └── drawable/ └── ic_launcher.xml
```

5. Referências

1. Android Developers. *Audio output for Wear OS*. Disponível em:
<https://developer.android.com/training/wearables/audio>
2. Android Developers. *AudioManager*. Disponível em:
<https://developer.android.com/reference/android/media/AudioManager>
3. Android Developers. *TextToSpeech*. Disponível em:
<https://developer.android.com/reference/android/speech/tts/TextToSpeech>
4. Android Developers. *Create a Wear OS project*. Disponível em:
<https://developer.android.com/training/wearables/get-started/creating>
5. Material de orientação — DGT2816, Estácio. Disponível em:
<https://sway.cloud.microsoft/d3rPssMprgwvvxGP>

DomaWearOS — Trabalho Prático | DGT2816 — Interação com sensores de smartphones e wearables

Estácio | Kotlin + Wear OS SDK | 2026